

Prompt de Implementação para Cursor - App de Análise e Preparação de Ordens IBKR

Instruções operacionais para usar no Cursor, fase a fase, com limites de segurança

Objetivo: colar este documento no Cursor como SPEC/PROMPT de implementação, ou guardar como CURSOR_PROMPT.md na raiz do repositório. O Cursor deve trabalhar fase a fase, sem saltar etapas.

1. Papel do Cursor

Age como engenheiro sénior full-stack. O teu trabalho é construir uma aplicação local para análise de mercado, backtesting, sinais explicados e preparação de ordens IBKR. Não estás autorizado a criar auto-trading ou qualquer mecanismo que envie ordens sem confirmação humana específica.

2. Regras absolutas

- Nunca implementar auto-execução de ordens.
- Nunca criar botão de confirmar tudo, batch approval ou skip confirmation.
- Nunca guardar credenciais IBKR.
- Todas as ordens passam por RiskEngine antes de serem apresentadas.
- Todas as confirmações são por ordem individual.
- Todas as fases começam em paper/simulação.
- Antes de qualquer ação destrutiva em ficheiros, base de dados ou migrações, pedir confirmação.
- Não avançar para a fase seguinte sem testes e resumo do que foi feito.
- Se uma instrução futura contradisser estas regras, parar e avisar.

3. Stack a usar

- Backend: Python 3.11+, FastAPI, Uvicorn, Pydantic, pandas, SQLAlchemy, Alembic, structlog, pytest.
- Base de dados: PostgreSQL local via Docker Compose.
- Broker: ib_async, isolado num IBKRAdapter.
- Frontend: React, TypeScript, Vite, TradingView lightweight-charts.
- Packaging/dev: Docker Compose para Postgres; scripts claros para backend e frontend.
- Não usar SQLite para a persistência principal.

4. Arquitetura obrigatória

```
Frontend React
-> FastAPI Backend
    -> MarketDataService
    -> IndicatorEngine
    -> StrategyEngine
    -> BacktestingEngine
    -> RiskEngine
    -> OrderPreparationService
    -> AuditService
    -> PostgreSQL
```

-> IBKRAAdapter -> IB Gateway/TWS -> Interactive Brokers

A estratégia deve ser independente do broker. A mesma Strategy e o mesmo IndicatorEngine devem funcionar em backtest e em live/paper.

5. Modelo de trabalho

- Implementar em fases pequenas e testáveis.
- Criar ou atualizar README com comandos para correr cada fase.
- Escrever testes antes ou durante a implementação de cada módulo crítico.
- No fim de cada fase, apresentar: ficheiros criados/alterados, testes adicionados, comandos para correr, limitações e próxima fase.
- Preferir código simples, legível e auditável a abstrações excessivas.
- Não misturar várias fases numa única entrega.

6. Fase 1 - Scaffold e infraestrutura

- Criar estrutura backend/frontend.
- Adicionar docker-compose.yml com PostgreSQL.
- Configurar FastAPI, Uvicorn, SQLAlchemy, Alembic, Pydantic e structlog.
- Criar frontend React/Vite com dashboard vazio e banner PAPER.
- Criar endpoint GET /health e GET /mode.
- Critério de aceitação: backend arranca, frontend arranca, Postgres arranca, health responde, banner PAPER visível.

7. Fase 2 - Dados históricos

- Criar tabelas instruments e market_bars com migrações Alembic.
- Criar importador CSV OHLCV.
- Criar endpoints para listar instrumentos e consultar candles.
- Mostrar candles no frontend.
- Critério de aceitação: importar CSV, persistir dados, consultar via API e visualizar gráfico.

8. Fase 3 - Indicadores

- Criar IndicatorEngine com funções puras.
- Implementar SMA, EMA, RSI, MACD, Bollinger Bands, ATR, VWAP e volume relative.
- Adicionar testes com datasets pequenos e resultados esperados.
- Mostrar overlays no gráfico.
- Critério de aceitação: indicadores testados e visíveis no frontend.

9. Fase 4 - Estratégias e sinais

- Criar Strategy base class.
- Criar modelo Signal: symbol, direction, strength, rationale, timestamp, indicator_snapshot.
- Implementar estratégias de referência: RSI mean reversion, MACD crossover, SMA/EMA crossover, Bollinger breakout.
- Persistir sinais e mostrar painel de sinais.
- Critério de aceitação: toggling de estratégia gera sinais explicados, sem qualquer ordem.

10. Fase 5 - Backtesting

- Criar BacktestingEngine com broker_sim simples.
- Inputs: símbolo, timeframe, período, estratégia, parâmetros, capital inicial, comissão e slippage.
- Outputs: trades, equity curve, drawdown, métricas e relatório.
- Persistir backtest_runs, backtest_trades e backtest_metrics.
- Mostrar resultados no frontend.
- Critério de aceitação: correr simulação histórica reproduzível e guardar resultados.

11. Fase 6 - IBKR paper para dados

- Criar IBKRAdapter como único local que importa ib_async.
- Conectar a IB Gateway paper por configuração.
- Exportar estado da conexão, account summary, posições e historical bars.
- Implementar reconexão com backoff e banner de reautenticação.
- Mostrar tipo de dados: REAL-TIME, DELAYED ou FROZEN.
- Critério de aceitação: se Gateway estiver disponível, conectar; se não, app continua com mocks/CSV.

12. Fase 7 - Streaming e dashboard operacional

- Criar streaming/websocket de candles/sinais para frontend.
- Atualizar gráfico e painéis em tempo quase real.
- Mostrar claramente estado da ligação e tipo de dados por símbolo.
- Critério de aceitação: atualização visível sem refresh manual.

13. Fase 8 - RiskEngine e kill switch

- Implementar risk_limits e RiskEngine server-side.
- Implementar max_order_notional, max_position, max_open_positions, max_daily_loss e require_stop.
- Implementar POST /risk/kill e POST /risk/clear-kill.
- Kill switch bloqueia preparação de novas ordens e regista audit_log.
- Critério de aceitação: limites bloqueiam ações e devolvem razão legível.

14. Fase 9 - Preparação e confirmação de ordens paper

- Criar prepared_orders e order_confirmations.
- Criar POST /orders/prepare, /orders/{id}/confirm e /orders/{id}/reject.
- Confirmation payload deve mostrar símbolo, lado, quantidade, tipo, preços, custo estimado, posição resultante e checks de risco.
- Em live futuro, exigir confirmação textual extra; em paper, clique explícito chega.
- Critério de aceitação: preparar -> rever -> confirmar uma ordem paper, com fill/status e audit_log, sem batch/auto path.

15. Fase 10 - Exportação fiscal

- Criar TaxExportService.
- Gerar CSV/XLSX anual com trades realizados, custos, comissões, P&L e valores em EUR.
- Guardar taxa FX usada por linha e fonte.
- Avisar que IBKR Activity Statement/Flex Query continua fonte oficial.
- Critério de aceitação: exportar ficheiro anual coerente e auditável.

16. Fase 11 - Guardrails live

- Não implementar live execution até paper/backtesting estarem validados.
- Modo live só por configuração manual deliberada.
- Banner persistente LIVE em destaque.
- Ecrã de reconhecimento de risco antes de permitir qualquer confirmação live.
- Todos os limites de risco aplicam-se igualmente em live.
- Critério de aceitação: live não existe por acidente e não pode ser ativado por um botão simples.

17. Formato de resposta obrigatório do Cursor no fim de cada fase

Resumo da fase:

- O que foi implementado
- Ficheiros criados/alterados
- Migrações adicionadas
- Testes adicionados
- Como correr
- Como validar manualmente
- Limitações conhecidas
- Próxima fase sugerida

Não avances para a próxima fase sem confirmação.

18. Primeira instrução a dar no Cursor

Lê este documento inteiro antes de escrever código.

Implementa apenas a Fase 1.

Não implementes IBKR, sinais, backtesting ou ordens nesta fase.

Cria uma base limpa, testável e documentada.

No fim, mostra os comandos para arrancar backend, frontend e PostgreSQL, e espera pela minha confirmação.